RPG (POO + SOLID + outros paradigmas)

Descrição

Cada personagem possui um nome, uma quantidade de pontos de vida e um valor base de ataque.

Durante o combate, qualquer personagem pode atacar outro alvo ou receber dano, além de poder mostrar seu status atual.

Existem três tipos especiais de combatentes:

1. O guerreiro, que entra em combate armado e pode usar uma habilidade especial chamada 'Golpe Brutal' para causar dano extra.
2. O mago, que além dos atributos comuns também possui uma reserva de mana[1] (pontos de magia); ele pode usar uma habilidade especial chamada 'Bola de Fogo', gastando seus pontos mágicos para causar dano elevado.
3. O inimigo (como um goblin), que age como um adversário comum enfrentando os heróis.

Quando um combatente sofre dano suficiente para zerar sua vida, ele é derrotado.

Identificando candidatos a classes, atributos e métodos

Técnica clássica de Abbott para extrair candidatos a classes, atributos e métodos.

Substantivos comuns -> Classes ou Atributos
Substantivos próprios -> Objetos
Verbos ou locuções verbais -> Métodos

Cada **personagem** possui um **nome**, uma **quantidade de pontos de vida** e um **valor base de ataque**.

Durante o combate, qualquer **personagem** pode **atacar outro alvo** ou **receber dano**, além de poder **mostrar seu status atual**.

Existem três tipos especiais de combatentes: (Opa… isso tem cheiro de herança)

4. O **guerreiro**, que entra em combate armado e pode **usar uma habilidade especial chamada 'Golpe Brutal'** para **causar dano extra**.
5. O **mago**, que além dos atributos comuns também possui uma **reserva de mana**[2] (pontos de magia); ele pode **usar uma habilidade especial chamada 'Bola de Fogo'**, gastando seus pontos mágicos para causar dano elevado.
6. O **inimigo** (como um **goblin**), que **age** como um **adversário** comum **enfrentando** os **heróis**.

Quando um **combatente** **sofre dano** suficiente para zerar sua **vida**, **ele** **é derrotado**.

[1] A energia para lançar feitiços ou usar habilidades especiais.
[2] A energia para lançar feitiços ou usar habilidades especiais.

Comentários

Alguém possui algo => classe : atributo
Guerreiro é um Personagem. Mago é um Personagem. Inimigo é um Personagem.
Mago tem atributos semelhantes com outros personagens, mas tem atributo específico (o mana).
Novamente o "possui" sugerindo uma relação entre classe e atributo
"usar habilidade especial" sugere um método (por ser verbo) e aparece com nomes diferentes sugerindo comportamentos diferentes: - No guerreiro, consome energia de ataque; - No mago, consome mana Isso sugere um polimorfismo.
Então o inimigo é um personagem como qualquer outro. Talvez com mais vida para ser mais difícil de se matar. Talvez mais forte (com ataques poderosos). Talvez mais resistente (sofre menos danos). Talvez com habilidades especiais.



Diagrama de Classes

mermaid

None

```mermaid
classDiagram
    class Personagem {
        -nome String
        -vida int
        -ataque int
        +__init__(nome, vida, ataque)  %% Construtor da classe
        +atacar(alvo)
        +receber_dano(valor)
        +mostrar_status()
    }

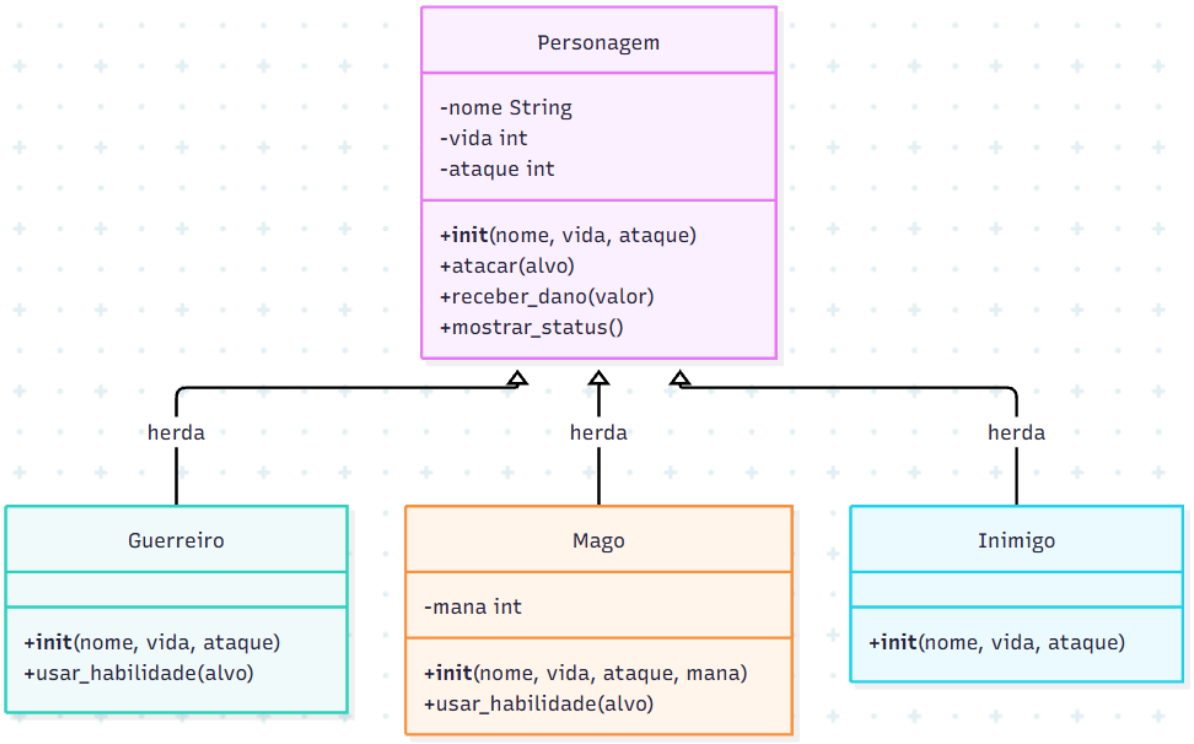
    class Guerreiro {
        +__init__(nome, vida, ataque)  %% Construtor da classe
        +usar_habilidade(alvo)
    }

    class Mago {
        -mana int
        +__init__(nome, vida, ataque, mana)
        +usar_habilidade(alvo)
    }

    class Inimigo {
        +__init__(nome, vida, ataque)
    }

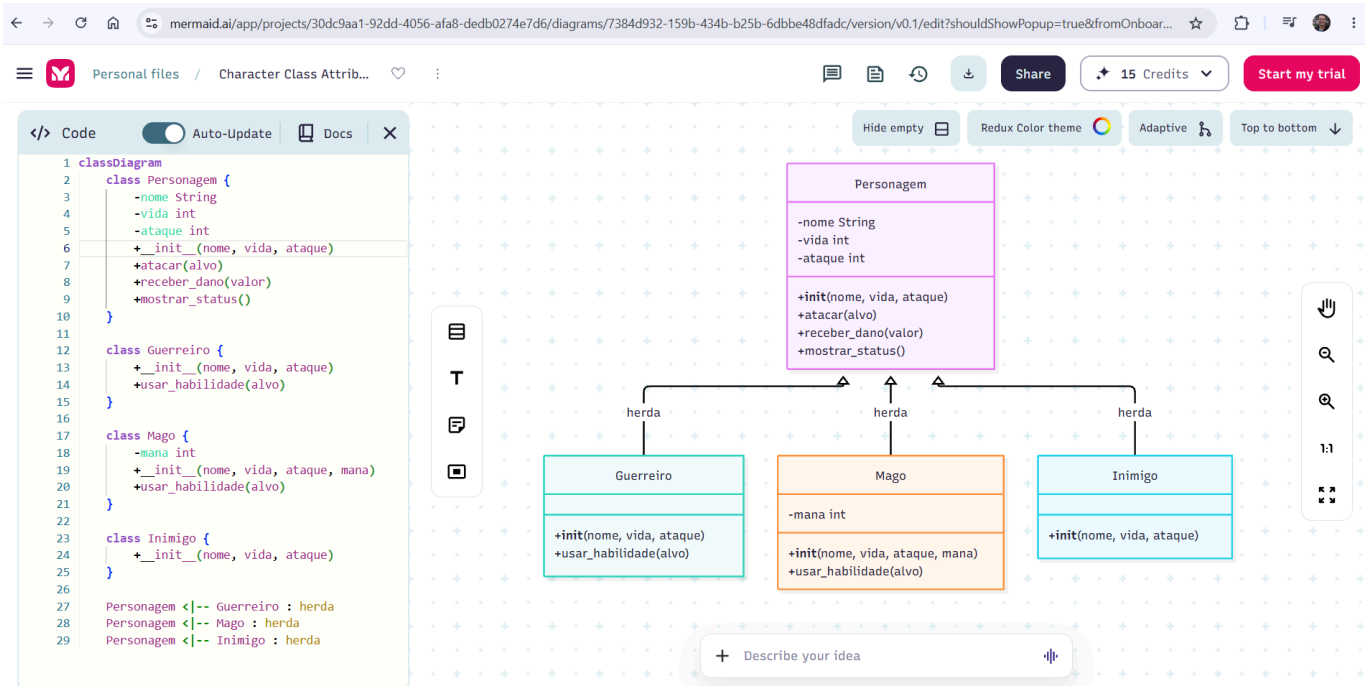
    Personagem <|-- Guerreiro : herda
    Personagem <|-- Mago : herda
    Personagem <|-- Inimigo : herda
```

O diagrama de classe em si (figura)



A imagem acima foi gerada em:

https://mermaid.ai/app/projects/30dc9aa1-92dd-4056-afa8-dedb0274e7d6/diagrams/7384d932-159b-434b-b25b-6dbbe48dfadc/version/v0.1/edit?shouldShowPopup=true&fromOnboarding=true



Java - version 01

Código

Java

```java
class Personagem {
    protected String nome;
    protected int vida;
    protected int ataque;

    public Personagem(String nome, int vida, int ataque) {
        this.nome = nome;
        this.vida = vida;
        this.ataque = ataque;
    }

    public void atacar(Personagem alvo) {
        System.out.println(this.nome + " ataca " + alvo.nome + "!");
        alvo.receberDano(this.ataque);
    }

    public void receberDano(int valor) {
        this.vida -= valor;
        System.out.println(this.nome + " recebeu " + valor + " de dano.");

        if (this.vida <= 0) {
            System.out.println(this.nome + " foi derrotado!");
        }
    }

    public void mostrarStatus() {
        System.out.println(this.nome + " - Vida: " + this.vida);
    }
}

class Guerreiro extends Personagem {
    public Guerreiro(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }

    public void usarHabilidade(Personagem alvo) {
        int dano = this.ataque + 5;
        System.out.println(this.nome + " usa Golpe Brutal!");
        alvo.receberDano(dano);
    }
}
```

```java
class Mago extends Personagem {
    private int mana;

    public Mago(String nome, int vida, int ataque, int mana) {
        super(nome, vida, ataque);
        this.mana = mana;
    }

    public void usarHabilidade(Personagem alvo) {
        if (this.mana > 0) {
            int dano = this.ataque + 10;
            this.mana -= 5;
            System.out.println(this.nome + " lança Bola de Fogo!");
            alvo.receberDano(dano);
        } else {
            System.out.println(this.nome + " não tem mana!");
        }
    }
}

class Inimigo extends Personagem {
    public Inimigo(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }
}

public class Main {
    public static void main(String[] args) {
        // Criando objetos
        Guerreiro heroi = new Guerreiro("Aragorn", 40, 8);
        Mago feiticeiro = new Mago("Luna", 30, 6, 20);
        Inimigo monstro = new Inimigo("Goblin", 25, 5);

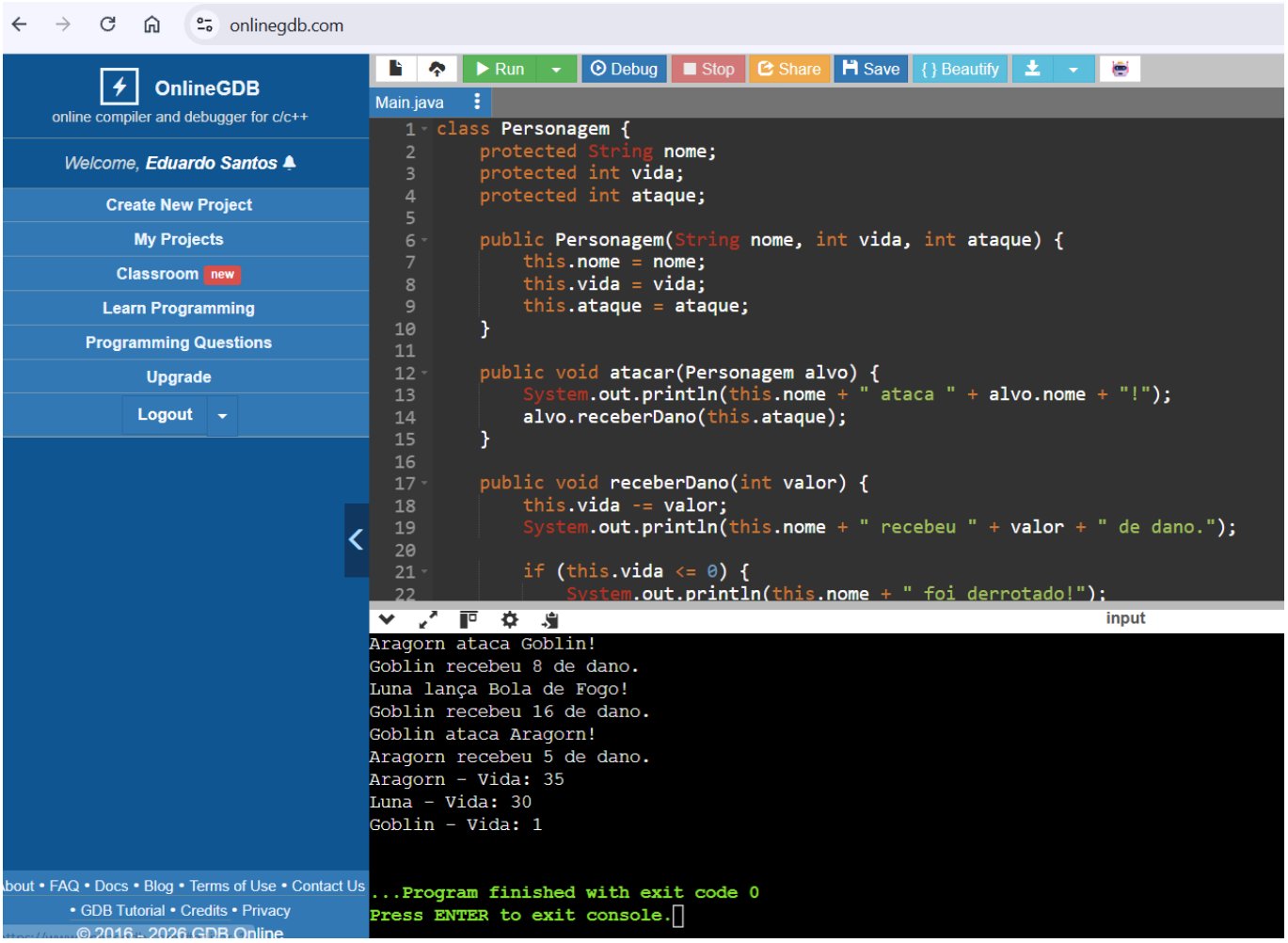
        // Batalha
        heroi.atacar(monstro);
        feiticeiro.usarHabilidade(monstro);
        monstro.atacar(heroi);

        heroi.mostrarStatus();
        feiticeiro.mostrarStatus();
        monstro.mostrarStatus();
    }
}
```

Execução

```
Aragorn ataca Goblin!
Goblin recebeu 8 de dano.
Luna lança Bola de Fogo!
Goblin recebeu 16 de dano.
Goblin ataca Aragorn!
Aragorn recebeu 5 de dano.
Aragorn - Vida: 35
Luna - Vida: 30
Goblin - Vida: 1
```

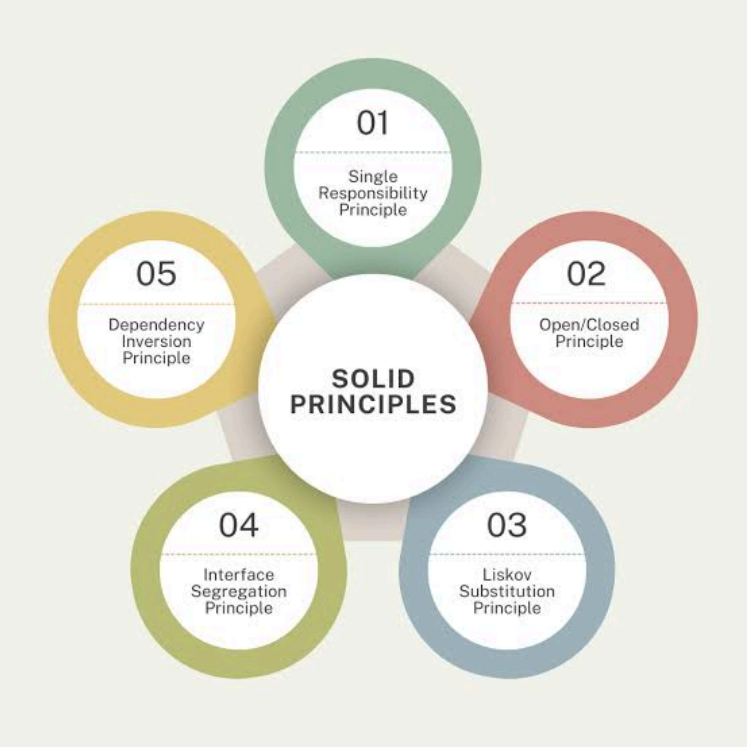
Executado no OnlineGDB (https://www.onlinegdb.com/)



SOLID - Análise do código quanto aos princípios

SOLID é um acrônimo para cinco princípios de design de código na **Programação Orientada a Objetos (POO)** criados para tornar os sistemas mais fáceis de manter, entender e expandir.

Os conceitos foram reunidos por Robert C. Martin (conhecido como Uncle Bob) para ajudar desenvolvedores a evitar códigos complexos e difíceis de alterar. Mas o acrônimo foi cunhado por outra pessoa.



Os 5 Princípios do SOLID

- **S — Single Responsibility Principle (Princípio da Responsabilidade Única):** Uma classe deve ter um, e apenas um, motivo para mudar. Em suma, cada classe deve cuidar de apenas uma parte específica da funcionalidade do sistema.
- **O — Open-Closed Principle (Princípio Aberto-Fechado):** As entidades de software (como classes e módulos) devem estar abertas para extensão, mas fechadas para modificação. Você deve conseguir adicionar novas funções sem alterar o código original que já está funcionando.
- **L — Liskov Substitution Principle (Princípio da Substituição de Liskov):** Objetos de uma classe derivada devem poder substituir objetos da sua classe base sem quebrar o funcionamento do programa. As subclasses precisam cumprir o contrato da classe mãe de forma correta.
- **I — Interface Segregation Principle (Princípio da Segregação de Interface):** É melhor criar várias interfaces específicas do que uma única interface genérica gigante. Nenhuma classe deve ser forçada a implementar métodos que não vai usar.
- **D — Dependency Inversion Principle (Princípio da Inversão de Dependência):** Você deve depender de abstrações (como interfaces), e não de implementações concretas. Isso reduz o acoplamento entre os módulos do sistema.

Violações do SRP (Single Responsibility Principle)

A classe Personagem apresenta várias responsabilidades distintas:

1. Regras de negócio: cálculo de dano e controle do ciclo de vida (vida -= valor) e if vida < 0, “se ferrou”.

2. Apresentação / I/O: formatação e exibição direta de mensagens no console via System.out.println.

3. Notificação de eventos: detecção imediata de derrota (Diz: “Morreu” assim na lata - nem dá tempo para um elixirzinho).

Portanto, ela viola o primeiro princípio (S) do SOLID. Alguns problemas que poderiam ocorrer no futuro:

- Se, numa nova versão, o jogo for implementado para ser executado em uma interface gráfica ou interface web etc., o uso de System.out.println(), que serve apenas para a execução em console torna-se completamente impróprio.
- Se, no futuro, nosso jogo ganhar o mundo, todos terão que entender o português, pois as mensagens estão escritas diretamente neste idioma. E se quisermos agora, soltar as versões em inglês e mandarim (né não Chen?)?
- Só o método receberDano tem, pelo menos quatro responsabilidades:
    - Atualizar o valor da vida (deveria ser a única responsabilidade dele).
    - Formatar e exibir uma mensagem (em português) e de um jeito fixo (se no futuro você tiver ideias para dar mensagens diferentes a cada morte para tornar o jogo mais divertido, terá que reescrever muita coisa (observe que os princípios S e O costumam operar em conjunto).
    - A saída em console (“Que jogo arcaico! Só dá pra rodar em console.”).
    - Avalia se o personagem morreu e define o que ocorre (só exibe uma mensagem informando o óbito). E se no futuro você decidir que o personagem ressuscita em outro ponto (muito comum), ou que, em um primeiro momento, o personagem só entra em coma por um tempo, podendo ser curado por um de seus companheiros (algo também muito comum)?

Violações do OCP (Open-Closed Principle)

Em geral, violações do SRP também implicam violações do OCP, pois, quando uma classe assume responsabilidades que não deveriam pertencer a ela (SRP violado), qualquer novo requisito ou variação futura obriga a abertura e alteração cirúrgica dessa mesma classe (OCP violado). Com efeito, alguns dos problemas que já mencionamos correspondem a violações dos dois primeiros princípios:

Problema 1:

SRP: A classe `Personagem` acumula regras de combate com exibição no terminal (`System.out.println`) e formatação de texto. O problema apontado é a sobrecarga de papéis num mesmo arquivo.

OCP: Se amanhã o jogo precisar de efeitos sonoros, log em arquivo, tradução para outros idiomas ou interface gráfica, seremos forçados a **abrir a classe `Personagem` para editá-la**. Uma classe que deveria estar **fechada para alteração** (pois as regras de dano não mudaram) precisa ser reaberta porque ela assumiu a responsabilidade da apresentação.

Problema 2:

SRP: Quando `receberDano` decide diretamente o que acontece na derrota:
(`if (vida <= 0) { ... derrotado! }`),
ele assume a responsabilidade do ciclo de vida global do combate.

OCP: O que acontece se introduzirmos um `Zumbi` que não morre na primeira vez que a vida zera? Ou um `Boss` que muda de fase aos 0 HP? Ou um herói com item de ressurreição?

- Teremos que adicionar blocos `if/else` ou `switch` dentro de `receberDano`.
- Essa centralização de casos específicos é o sintoma clássico de **código aberto para modificação**: para cada nova mecânica de derrota, mexe-se no método de cálculo de dano existente.

Violações do LSP (Liskov Substitution Principle)

Esse princípio diz que:

> *Objetos de uma classe derivada devem poder substituir objetos da sua classe base sem quebrar o funcionamento do programa. As subclasses precisam cumprir o contrato da classe mãe de forma correta.*

Onde ocorre esse problema no nosso código

Na hierarquia entre `Personagem`, `Inimigo`, `Guerreiro` e `Mago`.

Diagnóstico:

- `Personagem` pode ser instanciada diretamente, embora conceitualmente represente uma abstração incompleta.
- O `Inimigo` herda todos os métodos, mas não possui nenhuma habilidade especial, criando uma assimetria comportamental entre os tipos de combatentes.
- O polimorfismo fica quebrado para o chamador: você não pode tratar `Guerreiro`, `Mago` e `Inimigo` de maneira uniforme com relação a ações de ataque especial.

Suponha, por exemplo, que um monstro inimigo foi domado e passou a lutar ao lado dos guerreiros e magos contra outro grande e poderoso monstrão inimigo. Além disso, o programa permite formar uma lista de combatentes aliados para atacar algum alvo. Assim, em um dado momento, constrói-se a seguinte lista e loop que define o ataque coordenado dos aliados:

```
List<Personagem> aliados = List.of(heroi, feiticeiro, monstrinho);
for (Personagem p : aliados) {
            p.usarHabilidade(monstrao);
        }
```

Como o monstrinho não tem habilidade especial, esse código irá gerar erro.

Isso é justamente devido a um problema de violação do LSP: nem toda classe concreta que herda da classe pai tem todos os métodos implementados podendo vir a causar erros em cenários específicos como este ilustrado aqui.

Poderíamos corrigir isso forçando que todo personagem tenha todas as classes definidas na classe pai, mas isso implica forçarmos a implementação de métodos em classes que não precisam desses métodos (como criar métodos vazios), o que corresponde a violar outro princípio, o Interface Segregation Principle (ISP).

Como resolver isso? Segregando o contrato em mais de uma interface, ou distribuindo entre interfaces e classes abstratas.

Vejamos uma solução para o caso específico que estamos considerando.

Java

```java
import java.util.List;

// Base comum a TODOS os personagens (com ou sem habilidades)
abstract class Personagem {
    protected String nome;
    protected int vida;
    protected int ataque;

    public Personagem(String nome, int vida, int ataque) {
        this.nome = nome;
        this.vida = vida;
        this.ataque = ataque;
    }

    public void atacar(Personagem alvo) {
        System.out.println(this.nome + " ataca " + alvo.nome + "!");
        alvo.receberDano(this.ataque);
    }

    public void receberDano(int valor) {
        this.vida -= valor;
        System.out.println(this.nome + " recebeu " + valor + " de dano.");
        if (this.vida <= 0) {
            System.out.println(this.nome + " foi derrotado!");
        }
    }
```

```
    public void mostrarStatus() {
        System.out.println(this.nome + " - Vida: " + this.vida);
    }
}

// Contrato segregado: apenas para quem possui habilidades especiais
interface ComHabilidadeEspecial {
    void usarHabilidade(Personagem alvo);
}

// Guerreiro: É Personagem E possui Habilidade Especial
class Guerreiro extends Personagem implements ComHabilidadeEspecial {
    public Guerreiro(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }

    @Override
    public void usarHabilidade(Personagem alvo) {
        int dano = this.ataque + 5;
        System.out.println(this.nome + " usa Golpe Brutal!");
        alvo.receberDano(dano);
    }
}

// Mago: É Personagem E possui Habilidade Especial
class Mago extends Personagem implements ComHabilidadeEspecial {
    private int mana;

    public Mago(String nome, int vida, int ataque, int mana) {
        super(nome, vida, ataque);
        this.mana = mana;
    }

    @Override
    public void usarHabilidade(Personagem alvo) {
        if (this.mana >= 5) {
            int dano = this.ataque + 10;
            this.mana -= 5;
            System.out.println(this.nome + " lança Bola de Fogo!");
            alvo.receberDano(dano);
        } else {
            System.out.println(this.nome + " não tem mana!");
        }
    }
}

// Inimigo: É apenas um Personagem simples
class Inimigo extends Personagem {
    public Inimigo(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }
}
```

```java
// Início
public class Main {
    public static void main(String[] args) {
        Personagem heroi = new Guerreiro("Mike", 40, 8);
        Personagem feiticeira = new Mago("Eleven", 30, 6, 20);
        Personagem monstrinho = new Inimigo("Dart", 25, 5);
        Personagem monstrao = new Inimigo("Demogorgon", 100, 50);

        List<Personagem> aliados = List.of(heroi, feiticeira, monstrinho);

        // Opção 1: Executando apenas em quem implementa a interface de habilidade
        for (Personagem p : aliados) {
            if (p instanceof ComHabilidadeEspecial especial) {
                especial.usarHabilidade(monstrao);
            } else {
                p.atacar(monstrao); // Personagens sem habilidade usam o ataque padrão
            }
        }

        monstrao.mostrarStatus();
    }
}
```

O **instanceof** em Java é um operador binário usado para verificar se um objeto é uma instância de uma classe específica, de uma subclasse ou se implementa uma interface.

Essa modelagem preserva simultaneamente:

1. **LSP:** Todo `Personagem` cumpre 100% das promessas de `Personagem`.
2. **ISP:** Nenhuma subclasse herda métodos artificiais que não fazem sentido para o seu domínio.
3. **OCP:** Novos tipos (ex.: `Arqueiro implements ComHabilidadeEspecial` ou `Zumbi extends Personagem`) são adicionados sem alterar nenhuma linha das classes existentes.

Violações do DIP (Dependency Inversion Principle)

Outra análise quanto à classe Personagem... nela define-se que atacar() tem como argumento um outro Personagem. Mas e se quisermos usar o mesmo método atacar() para desferir golpes algo inanimado como um baú de itens ou para destruir uma fortaleza ou veículo do adversário (tal como discutimos em sala)?

Uma ideia, conforme discutimos, seria termos uma interface **Destrutível** e fazer as classes de coisas que podem morrer ou serem destruídas (Personagem, Compartimentos, Construcoes, Veiculos) herdarem dela.

Java

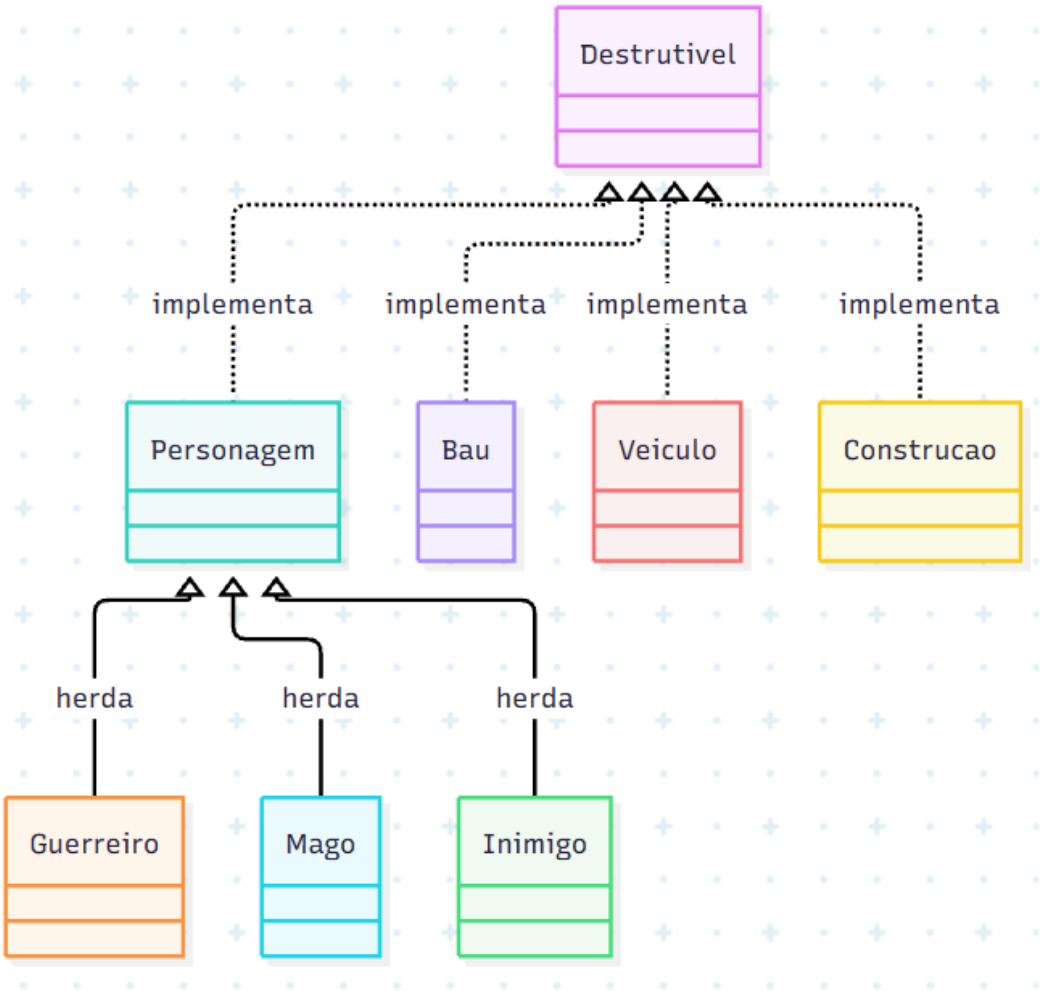
```java
public interface Destrutivel {
    void receberDano(int valor);
    boolean estaDestruido();
    String getNome();
}

public class Personagem implements Destrutivel {
    // ...
    public void atacar(Destrutivel alvo) { // Desacoplou de Personagem
        alvo.receberDano(this.ataque);
    }
}
```

Maior desacoplamento: atacar() agora depende de um interface, não de uma classe. Assim, podemos atacar qualquer coisa que assine o contrato da interface Destrutível, o que nos dá mais flexibilidade para acrescentar outros tipos de coisas atacáveis no futuro (criaturas, árvores, o chão etc.).

None

```
```mermaid
classDiagram
    Destrutivel <|.. Personagem : implementa
    Personagem <|-- Guerreiro : herda
    Personagem <|-- Mago : herda
    Personagem <|-- Inimigo : herda
    Destrutivel <|.. Bau : implementa
    Destrutivel <|.. Veiculo : implementa
    Destrutivel <|.. Construcao : implementa
```
```



Acrescentando o tratamento do DIP

Java

```java
import java.util.List;

// Abstração de capacidade de sofrer dano (Contrato DIP)
interface Destrutivel {
    void receberDano(int valor);
    String getNome();
}

// Base abstrata: Personagem é um Destrutivel e sabe atacar outros Destrutiveis
abstract class Personagem implements Destrutivel {
    protected String nome;
    protected int vida;
    protected int ataque;

    public Personagem(String nome, int vida, int ataque) {
        this.nome = nome;
        this.vida = vida;
        this.ataque = ataque;
    }

    // Desacoplado: aceita qualquer Destrutivel (monstro, baú, portão, etc.)
    public void atacar(Destrutivel alvo) {
```

```
        System.out.println(this.nome + " ataca " + alvo.getNome() + "!");
        alvo.receberDano(this.ataque);
    }

    @Override
    public void receberDano(int valor) {
        this.vida -= valor;
        System.out.println(this.nome + " recebeu " + valor + " de dano.");
        if (this.vida <= 0) {
            System.out.println(this.nome + " foi derrotado!");
        }
    }

    @Override
    public String getNome() {
        return this.nome;
    }

    public void mostrarStatus() {
        System.out.println(this.nome + " - Vida: " + this.vida);
    }
}

// Contrato segregado para quem domina técnicas especiais (também aceita Destrutivel)
interface ComHabilidadeEspecial {
    void usarHabilidade(Destrutivel alvo);
}

// Subclasses de Personagem
class Guerreiro extends Personagem implements ComHabilidadeEspecial {
    public Guerreiro(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }

    @Override
    public void usarHabilidade(Destrutivel alvo) {
        int dano = this.ataque + 5;
        System.out.println(this.nome + " usa Golpe Brutal em " + alvo.getNome() + "!");
        alvo.receberDano(dano);
    }
}

class Mago extends Personagem implements ComHabilidadeEspecial {
    private int mana;

    public Mago(String nome, int vida, int ataque, int mana) {
        super(nome, vida, ataque);
        this.mana = mana;
    }

    @Override
    public void usarHabilidade(Destrutivel alvo) {
        if (this.mana >= 5) {
            int dano = this.ataque + 10;
            this.mana -= 5;
            System.out.println(this.nome + " lança Bola de Fogo em " + alvo.getNome() + "!");
            alvo.receberDano(dano);
        } else {
            System.out.println(this.nome + " não tem mana!");
        }
```

```
    }
}

class Inimigo extends Personagem {
    public Inimigo(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }
}

// Entidades inanimadas: implementam Destrutivel sem herdar de Personagem
class Bau implements Destrutivel {
    private String nome;
    private int durabilidade;

    public Bau(String nome, int durabilidade) {
        this.nome = nome;
        this.durabilidade = durabilidade;
    }

    @Override
    public void receberDano(int valor) {
        this.durabilidade -= valor;
        System.out.println(this.nome + " sofreu " + valor + " de impacto.");
        if (this.durabilidade <= 0) {
            System.out.println(this.nome + " quebrou e revelou os itens!");
        }
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}

class Fortaleza implements Destrutivel {
    private String nome;
    private int blindagem;

    public Fortaleza(String nome, int blindagem) {
        this.nome = nome;
        this.blindagem = blindagem;
    }

    @Override
    public void receberDano(int valor) {
        // Reduz o dano sofrido devido à espessura da muralha
        int danoEfetivo = Math.max(1, valor - 3);
        this.blindagem -= danoEfetivo;
        System.out.println(this.nome + " absorveu impacto, sofrendo " + danoEfetivo + " de avaria.");
        if (this.blindagem <= 0) {
            System.out.println(this.nome + " ruiu em escombros!");
        }
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}
```

```java
// Execução do fluxo de jogo
public class Main {
    public static void main(String[] args) {
        // Instanciação de personagens e objetos inanimados
        Personagem heroi = new Guerreiro("Mike", 40, 8);
        Personagem feiticeira = new Mago("Eleven", 30, 6, 20);
        Personagem monstrinho = new Inimigo("Dart", 25, 5);
        Personagem monstrao = new Inimigo("Demogorgon", 50, 10);

        Destrutivel bauDeMadeira = new Bau("Baú Antigo", 12);
        Destrutivel portaoForte = new Fortaleza("Portão da Cidadela", 20);

        // Fase 1: Ataques a objetos inanimados do cenário
        System.out.println("--- Testando alvos do cenário ---");
        heroi.atacar(bauDeMadeira);
        feiticeira.atacar(portaoForte);

        if (heroi instanceof ComHabilidadeEspecial especial) {
            especial.usarHabilidade(portaoForte);
        }

        // Fase 2: Grupo de aliados enfrentando o monstrão
        System.out.println("\n--- Batalha contra o Chefe ---");
        List<Personagem> aliados = List.of(heroi, feiticeira, monstrinho);

        for (Personagem p : aliados) {
            if (p instanceof ComHabilidadeEspecial especial) {
                especial.usarHabilidade(monstrao);
            } else {
                p.atacar(monstrao);
            }
        }

        System.out.println("\n--- Status Final ---");
        monstrao.mostrarStatus();
    }
}
```

```
--- Testando alvos do cenário ---
Mike ataca Baú Antigo!
Baú Antigo sofreu 8 de impacto.
Eleven ataca Portão da Cidadela!
Portão da Cidadela absorveu impacto, sofrendo 3 de avaria.
Mike usa Golpe Brutal em Portão da Cidadela!
Portão da Cidadela absorveu impacto, sofrendo 10 de avaria.

--- Batalha contra o Chefe ---
Mike usa Golpe Brutal em Demogorgon!
Demogorgon recebeu 13 de dano.
Eleven lança Bola de Fogo em Demogorgon!
Demogorgon recebeu 16 de dano.
Dart ataca Demogorgon!
Demogorgon recebeu 5 de dano.

--- Status Final ---
Demogorgon - Vida: 16
```

TOTAL SOLID - Código refatorado (pelo Gemini) para atender totalmente ao SOLID

Java

```
import java.text.MessageFormat;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// ========================================
// 1. CAMADA DE INTERNACIONALIZAÇÃO (i18n)
// ========================================
interface MessageProvider {
    String getMessage(String key, Object... args);
}

abstract class MapBasedMessageProvider implements MessageProvider {
    protected final Map<String, String> messages = new HashMap<>();

    @Override
    public String getMessage(String key, Object... args) {
        String template = messages.getOrDefault(key, key);
        return MessageFormat.format(template, args);
    }
}

// Suporte a Português
class PortugueseMessageProvider extends MapBasedMessageProvider {
    public PortugueseMessageProvider() {
        messages.put("attack", "{0} ataca {1}!");
        messages.put("damage_received", "{0} recebeu {1} de dano.");
        messages.put("defeated", "{0} foi derrotado!");
        messages.put("status", "{0} - Vida: {1}");
        messages.put("brutal_strike", "{0} usa Golpe Brutal em {1}!");
        messages.put("fireball", "{0} lança Bola de Fogo em {1}!");
        messages.put("no_mana", "{0} não tem mana!");
        messages.put("chest_damage", "{0} sofreu {1} de impacto.");
        messages.put("chest_broken", "{0} quebrou e revelou os itens!");
          messages.put("fortress_damage", "{0} absorveu impacto, sofrendo {1}
de avaria.");
        messages.put("fortress_broken", "{0} ruiu em escombros!");
    }
}

// Suporte a Inglês
class EnglishMessageProvider extends MapBasedMessageProvider {
    public EnglishMessageProvider() {
```

```
        messages.put("attack", "{0} attacks {1}!");
        messages.put("damage_received", "{0} took {1} damage.");
        messages.put("defeated", "{0} was defeated!");
        messages.put("status", "{0} - Health: {1}");
        messages.put("brutal_strike", "{0} uses Brutal Strike on {1}!");
        messages.put("fireball", "{0} casts Fireball on {1}!");
        messages.put("no_mana", "{0} does not have enough mana!");
        messages.put("chest_damage", "{0} suffered {1} impact damage.");
        messages.put("chest_broken", "{0} broke open, revealing loot!");
           messages.put("fortress_damage", "{0} absorbed the hit, taking {1}
structural damage.");
        messages.put("fortress_broken", "{0} collapsed into rubble!");
    }
}

// Suporte a Mandarim (Chinês Simplificado)
class ChineseMessageProvider extends MapBasedMessageProvider {
    public ChineseMessageProvider() {
        messages.put("attack", "{0} 攻击了 {1}！");
        messages.put("damage_received", "{0} 受到了 {1} 点伤害。");
        messages.put("defeated", "{0} 被击败了！");
        messages.put("status", "{0} - 生命值: {1}");
        messages.put("brutal_strike", "{0} 对 {1} 施展了野蛮打击！");
        messages.put("fireball", "{0} 对 {1} 发射了火球术！");
        messages.put("no_mana", "{0} 法力值不足！");
        messages.put("chest_damage", "{0} 承受了 {1} 点冲击。");
        messages.put("chest_broken", "{0} 被击碎，露出了里面的宝物！");
          messages.put("fortress_damage", "{0} 吸收了冲击，承受了 {1} 点结构损毁。
");
        messages.put("fortress_broken", "{0} 轰然倒塌化作废墟！");
    }
}

// ============================================
// 2. CAMADA DE SAÍDA / APRESENTAÇÃO (I/O)
// ============================================
interface OutputChannel {
    void print(String message);
}

class ConsoleOutputChannel implements OutputChannel {
    @Override
    public void print(String message) {
        System.out.println(message);
    }
```

```java
}

// Implementação alternativa para GUI ou Testes Unitários
class LogInMemoryOutputChannel implements OutputChannel {
    private final StringBuilder log = new StringBuilder();

    @Override
    public void print(String message) {
        log.append(message).append("\n");
    }

    public String getLog() {
        return log.toString();
    }
}

// Mediador/Notificador de Eventos (Isola domínio de apresentação)
class CombatNotifier {
    private final OutputChannel output;
    private final MessageProvider messages;

    public CombatNotifier(OutputChannel output, MessageProvider messages) {
        this.output = output;
        this.messages = messages;
    }

    public void notify(String key, Object... args) {
        String msg = messages.getMessage(key, args);
        output.print(msg);
    }
}

// ========================================
// 3. CAMADA DE DOMÍNIO (Regras de Combate)
// ========================================
interface Destrutivel {
    void receberDano(int valor, CombatNotifier notifier);
    boolean estaDestruido();
    String getNome();
}

abstract class Personagem implements Destrutivel {
    protected String nome;
    protected int vida;
    protected int ataque;
```

```java
    public Personagem(String nome, int vida, int ataque) {
        this.nome = nome;
        this.vida = vida;
        this.ataque = ataque;
    }

    public void atacar(Destrutivel alvo, CombatNotifier notifier) {
        notifier.notify("attack", this.nome, alvo.getNome());
        alvo.receberDano(this.ataque, notifier);
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.vida = Math.max(0, this.vida - valor);
        notifier.notify("damage_received", this.nome, valor);
        if (estaDestruido()) {
            notifier.notify("defeated", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.vida == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }

    public void mostrarStatus(CombatNotifier notifier) {
        notifier.notify("status", this.nome, this.vida);
    }
}

interface ComHabilidadeEspecial {
    void usarHabilidade(Destrutivel alvo, CombatNotifier notifier);
}

class Guerreiro extends Personagem implements ComHabilidadeEspecial {
    public Guerreiro(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }
```

```
    @Override
    public void usarHabilidade(Destrutivel alvo, CombatNotifier notifier) {
        int dano = this.ataque + 5;
        notifier.notify("brutal_strike", this.nome, alvo.getNome());
        alvo.receberDano(dano, notifier);
    }
}

class Mago extends Personagem implements ComHabilidadeEspecial {
    private int mana;

    public Mago(String nome, int vida, int ataque, int mana) {
        super(nome, vida, ataque);
        this.mana = mana;
    }

    @Override
    public void usarHabilidade(Destrutivel alvo, CombatNotifier notifier) {
        if (this.mana >= 5) {
            int dano = this.ataque + 10;
            this.mana -= 5;
            notifier.notify("fireball", this.nome, alvo.getNome());
            alvo.receberDano(dano, notifier);
        } else {
            notifier.notify("no_mana", this.nome);
        }
    }
}

class Inimigo extends Personagem {
    public Inimigo(String nome, int vida, int ataque) {
        super(nome, vida, ataque);
    }
}

// Entidades inanimadas
class Bau implements Destrutivel {
    private final String nome;
    private int durabilidade;

    public Bau(String nome, int durabilidade) {
        this.nome = nome;
        this.durabilidade = durabilidade;
    }
```

```java
    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.durabilidade = Math.max(0, this.durabilidade - valor);
        notifier.notify("chest_damage", this.nome, valor);
        if (estaDestruido()) {
            notifier.notify("chest_broken", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.durabilidade == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}

class Fortaleza implements Destrutivel {
    private final String nome;
    private int blindagem;

    public Fortaleza(String nome, int blindagem) {
        this.nome = nome;
        this.blindagem = blindagem;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        int danoEfetivo = Math.max(1, valor - 3);
        this.blindagem = Math.max(0, this.blindagem - danoEfetivo);
        notifier.notify("fortress_damage", this.nome, danoEfetivo);
        if (estaDestruido()) {
            notifier.notify("fortress_broken", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.blindagem == 0;
    }

    @Override
```

```
    public String getNome() {
        return this.nome;
    }
}

// ========================================
// 4. PROGRAMA PRINCIPAL
// ========================================
public class Main {

    private static void executarBatalha(CombatNotifier notifier) {
        Personagem heroi = new Guerreiro("Mike", 40, 8);
        Personagem feiticeira = new Mago("Eleven", 30, 6, 20);
        Personagem monstrinho = new Inimigo("Dart", 25, 5);
        Personagem monstrao = new Inimigo("Demogorgon", 45, 10);

        Destrutivel bau = new Bau("Baú Antigo", 10);
        Destrutivel portao = new Fortaleza("Portão da Cidadela", 15);

        // Cenário 1: Ataque a objetos
        heroi.atacar(bau, notifier);
        feiticeira.atacar(portao, notifier);

        // Cenário 2: Batalha em grupo contra o monstro
        List<Personagem> aliados = List.of(heroi, feiticeira, monstrinho);
        for (Personagem p : aliados) {
            if (p instanceof ComHabilidadeEspecial especial) {
                especial.usarHabilidade(monstrao, notifier);
            } else {
                p.atacar(monstrao, notifier);
            }
        }

        monstrao.mostrarStatus(notifier);
    }

    public static void main(String[] args) {
        OutputChannel console = new ConsoleOutputChannel();

        System.out.println("========== TESTE EM PORTUGUÊS ==========");
              CombatNotifier ptNotifier = new CombatNotifier(console, new
PortugueseMessageProvider());
        executarBatalha(ptNotifier);

        System.out.println("\n========== TEST IN ENGLISH ==========");
```

```
            CombatNotifier enNotifier = new CombatNotifier(console, new
EnglishMessageProvider());
        executarBatalha(enNotifier);

        System.out.println("\n========== 中文测试 (MANDARIM) ==========");
            CombatNotifier zhNotifier = new CombatNotifier(console, new
ChineseMessageProvider());
        executarBatalha(zhNotifier);
    }
}
```

```
========== TESTE EM PORTUGUÊS ==========
Mike ataca Baú Antigo!
Baú Antigo sofreu 8 de impacto.
Eleven ataca Portão da Cidadela!
Portão da Cidadela absorveu impacto, sofrendo 3 de avaria.
Mike usa Golpe Brutal em Demogorgon!
Demogorgon recebeu 13 de dano.
Eleven lança Bola de Fogo em Demogorgon!
Demogorgon recebeu 16 de dano.
Dart ataca Demogorgon!
Demogorgon recebeu 5 de dano.
Demogorgon - Vida: 11

========== TEST IN ENGLISH ==========
Mike attacks Baú Antigo!
Baú Antigo suffered 8 impact damage.
Eleven attacks Portão da Cidadela!
Portão da Cidadela absorbed the hit, taking 3 structural damage.
Mike uses Brutal Strike on Demogorgon!
Demogorgon took 13 damage.
Eleven casts Fireball on Demogorgon!
Demogorgon took 16 damage.
Dart attacks Demogorgon!
Demogorgon took 5 damage.
Demogorgon - Health: 11

========== 中文测试 (MANDARIM) ==========
Mike 攻击了 Baú Antigo!
Baú Antigo 承受了 8 点冲击。
Eleven 攻击了 Portão da Cidadela!
Portão da Cidadela 吸收了冲击，承受了 3 点结构损毁。
Mike 对 Demogorgon 施展了野蛮打击!
Demogorgon 受到了 13 点伤害。
Eleven 对 Demogorgon 发射了火球术!
Demogorgon 受到了 16 点伤害。
Dart 攻击了 Demogorgon!
Demogorgon 受到了 5 点伤害。
Demogorgon - 生命值: 11
```

Arquitetura da Solução

1. **SRP (Responsabilidade Única):**
    - As entidades (`Personagem`, `Bau`, etc.) cuidam exclusivamente de seus atributos numéricos e regras de estado.
    - `MessageProvider` cuida apenas da tradução e formatação das mensagens.
    - `OutputChannel` cuida unicamente do destino dos dados (Console, Janela Gráfica, Buffer em Memória, etc.).
    - `GameEventNotifier` conecta as ações de domínio à exibição, sem acoplar as entidades à tela.
2. **OCP (Aberto/Fechado):**
    - Novos idiomas são adicionados criando uma nova classe que implemente `MessageProvider` (sem tocar no código existente).
    - Novos canais de saída (ex.: JavaFX, log em arquivo, socket web) são criados implementando `OutputChannel`.
3. **LSP (Substituição de Liskov):**
    - `Personagem` e suas subclasses podem ser substituídos sem surpresas ou quebras de contrato.
4. **ISP (Segregação de Interfaces):**
    - Contratos pequenos e focados: `Destrutivel`, `ComHabilidadeEspecial`, `OutputChannel` e `MessageProvider`.
5. **DIP (Inversão de Dependência):**
    - O sistema todo depende de interfaces, e a injeção do canal e do provedor de idiomas é feita de fora para dentro.

O diagrama de classe

None

```
classDiagram
    %% ============================================
    %% 1. CAMADA DE INTERNACIONALIZAÇÃO (i18n)
    %% ============================================
    class MessageProvider {
        <<interface>>
        +getMessage(key: String, args: Object[]) String
    }

    class MapBasedMessageProvider {
        <<abstract>>
        #Map~String, String~ messages
        +getMessage(key: String, args: Object[]) String
    }
```

```
class PortugueseMessageProvider {
    +PortugueseMessageProvider()
}

class EnglishMessageProvider {
    +EnglishMessageProvider()
}

class ChineseMessageProvider {
    +ChineseMessageProvider()
}

MessageProvider <|.. MapBasedMessageProvider : implements
MapBasedMessageProvider <|-- PortugueseMessageProvider : extends
MapBasedMessageProvider <|-- EnglishMessageProvider : extends
MapBasedMessageProvider <|-- ChineseMessageProvider : extends

%% ========================================
%% 2. CAMADA DE APRESENTAÇÃO (I/O)
%% ========================================
class OutputChannel {
    <<interface>>
    +print(message: String) void
}

class ConsoleOutputChannel {
    +print(message: String) void
}

class LogInMemoryOutputChannel {
    -StringBuilder log
    +print(message: String) void
    +getLog() String
}

class CombatNotifier {
    -OutputChannel output
    -MessageProvider messages
    +CombatNotifier(output: OutputChannel, messages: MessageProvider)
```

```
    +notify(key: String, args: Object[]) void
}

OutputChannel <|.. ConsoleOutputChannel : implements
OutputChannel <|.. LogInMemoryOutputChannel : implements

CombatNotifier --> OutputChannel : uses
CombatNotifier --> MessageProvider : uses

%% ========================================
%% 3. CAMADA DE DOMÍNIO (Regras de Combate)
%% ========================================
class Destrutivel {
    <<interface>>
    +receberDano(valor: int, notifier: CombatNotifier) void
    +estaDestruido() boolean
    +getNome() String
}

class ComHabilidadeEspecial {
    <<interface>>
    +usarHabilidade(alvo: Destrutivel, notifier: CombatNotifier) void
}

class Personagem {
    <<abstract>>
    #String nome
    #int vida
    #int ataque
    +Personagem(nome: String, vida: int, ataque: int)
    +atacar(alvo: Destrutivel, notifier: CombatNotifier) void
    +receberDano(valor: int, notifier: CombatNotifier) void
    +estaDestruido() boolean
    +getNome() String
    +mostrarStatus(notifier: CombatNotifier) void
}

class Guerreiro {
    +Guerreiro(nome: String, vida: int, ataque: int)
```

```
    +usarHabilidade(alvo: Destrutivel, notifier: CombatNotifier) void
}

class Mago {
    -int mana
    +Mago(nome: String, vida: int, ataque: int, mana: int)
    +usarHabilidade(alvo: Destrutivel, notifier: CombatNotifier) void
}

class Inimigo {
    +Inimigo(nome: String, vida: int, ataque: int)
}

class Bau {
    -String nome
    -int durabilidade
    +Bau(nome: String, durabilidade: int)
    +receberDano(valor: int, notifier: CombatNotifier) void
    +estaDestruido() boolean
    +getNome() String
}

class Fortaleza {
    -String nome
    -int blindagem
    +Fortaleza(nome: String, blindagem: int)
    +receberDano(valor: int, notifier: CombatNotifier) void
    +estaDestruido() boolean
    +getNome() String
}

Destrutivel <|.. Personagem : implements
Destrutivel <|.. Bau : implements
Destrutivel <|.. Fortaleza : implements

Personagem <|-- Guerreiro : extends
Personagem <|-- Mago : extends
Personagem <|-- Inimigo : extends
```

```
ComHabilidadeEspecial <|.. Guerreiro : implements
ComHabilidadeEspecial <|.. Mago : implements

%% Relacionamentos de dependência e uso no combate
Personagem ..> Destrutivel : ataca
Personagem ..> CombatNotifier : notifica
ComHabilidadeEspecial ..> Destrutivel : golpeia
ComHabilidadeEspecial ..> CombatNotifier : notifica
Bau ..> CombatNotifier : notifica
Fortaleza ..> CombatNotifier : notifica
```

https://mermaid.ai/app/projects/30dc9aa1-92dd-4056-afa8-dedb0274e7d6/diagrams/7384d932-159b-434b-b25b-6dbbe48dfadc/version/v0.1/edit?shouldShowPopup=true&fromOnboarding=true

Destaques das relações no diagrama

- **Realização (`<|..`):** `Personagem`, `Bau` e `Fortaleza` realizam a interface `Destrutivel`, permitindo que qualquer um seja alvo de ataques sem herança imprópria.
- **Segregação de Interfaces:** Apenas `Guerreiro` e `Mago` realizam `ComHabilidadeEspecial`, mantendo `Inimigo` enxuto e livre de métodos irrelevantes.
- **Inversão de Dependências (`..>`):** O domínio não referencia classes concretas de saída (`System.out`) nem dicionários de idioma específicos; todas as entidades interagem apenas através da abstração `CombatNotifier`, que por sua vez compõe `OutputChannel` e `MessageProvider`.

Uma **relação de dependência** (representada em UML por uma seta tracejada: `..>`) expressa a forma mais fraca e transitória de acoplamento entre duas classes.

Dizemos que a classe **A depende de B** quando **A utiliza B temporariamente**, de modo que qualquer alteração na interface pública ou comportamento de **B** pode forçar uma modificação em **A**.

Acrescentando associações

Associações são relações entre objetos que designam que um objeto possui ou contém outro. As associações são subdivididas em:

- **Agregação**: quando um objeto possui outro. Por exemplo, um Personagem pode ter uma armadura ou uma arma, que podem ser adquiridos, deixados ou destruídos. Quando o Personagem é destruído, o utensílio (arma ou armadura) continua existindo e pode ser adquirido por outro Personagem. Diz-se que o “todo” possui uma “parte”. Indica uma relação fraca onde a parte pode existir de forma independente do todo.

- **Composição**: quando um objeto contém outro. Por exemplo, um Personagem contém membros (braços, pernas, caudas) ou olhos - esses objetos também podem ser destruídos (decepando-se membros) ou arrancando-se um olho. Mas agora quando o Personagem é destruído, os objetos que ele contém (seus membros e órgãos) também o são. Diz-se que o “todo” contém uma “parte”. Indica uma relação forte e inseparável, onde o ciclo de vida da parte depende do todo.

Representação no diagrama de classes (e no mermaid):

- **Agregação:** É representada um losango aberto/branco no lado do "todo" (em mermaid, utiliza-se `o--` para denotar uma agregação).

- **Composição:** É representada um losango preenchido/preto no lado do "todo" (em mermaid utiliza-se `*--` para denotar uma composição.

Não vou incluir essas modificações no projeto atual, que já se tornou complexo demais. Para ficar mais didático, vou representar em um subprojeto menor.

A classe Membro (para a composição)

Java

```java
// A PARTE DA COMPOSIÇÃO
class Membro implements Destrutivel {
    private final String nome;
    private int integridade;

    public Membro(String nome, int integridade) {
        this.nome = nome;
        this.integridade = integridade;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.integridade = Math.max(0, this.integridade - valor);
        notifier.notify("damage_received", this.nome, valor);
        if (estaDestruido()) {
                notifier.notify("limb_severed", this.nome); // Notificação de
membro decepado!
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.integridade == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}
```

A classe Arma (para a agregação)

Java

```java
// A PARTE DA AGREGAÇÃO
class Arma implements Destrutivel {
    private final String nome;
    private final int bonusAtaque;
    private int durabilidade;

    public Arma(String nome, int bonusAtaque, int durabilidade) {
        this.nome = nome;
        this.bonusAtaque = bonusAtaque;
        this.durabilidade = durabilidade;
    }

    public int getBonusAtaque() {
        return this.bonusAtaque;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.durabilidade = Math.max(0, this.durabilidade - valor);
        if (estaDestruido()) {
            notifier.notify("weapon_broken", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.durabilidade == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}
```

A classe Personagem com as duas associações

Java

```
import java.util.ArrayList;
import java.util.List;

abstract class Personagem implements Destrutivel {
    protected String nome;
    protected int vida;
    protected int ataqueBase;

    // COMPOSIÇÃO: Ciclo de vida gerido aqui dentro
    // Quando o Personagem for destruído, seus membros também o são
    protected final List<Membro> membros = new ArrayList<>();

    // AGREGAÇÃO: Referência externa opcional,
    // não pertence exclusivamente ao ciclo de vida do Personagem
    protected Arma armaEquipada;

    public Personagem(String nome, int vida, int ataqueBase) {
        this.nome = nome;
        this.vida = vida;
        this.ataqueBase = ataqueBase;

        // COMPOSIÇÃO em ação: o Personagem cria suas próprias partes vitais
        this.membros.add(new Membro("Braço Direito de " + nome, 15));
        this.membros.add(new Membro("Braço Esquerdo de " + nome, 15));
        this.membros.add(new Membro("Perna Direita de " + nome, 20));
        this.membros.add(new Membro("Perna Esquerda de " + nome, 20));
    }

    // AGREGAÇÃO em ação: associar e desassociar sem destruir o objeto Arma
    public void equiparArma(Arma novaArma, CombatNotifier notifier) {
        this.armaEquipada = novaArma;
        notifier.notify("weapon_equipped", this.nome, novaArma.getNome());
    }

    public Arma droparArmaAoMorrer(CombatNotifier notifier) {
        if (this.armaEquipada != null) {
            Arma armaDropada = this.armaEquipada;
            this.armaEquipada = null; // Desassocia a arma
                                notifier.notify("weapon_dropped",   this.nome,
armaDropada.getNome());
                    return armaDropada; // A arma continua existindo para ser
recolhida!
        }
        return null;
```

```
    }

    public void atacar(Destrutivel alvo, CombatNotifier notifier) {
            int ataqueTotal = this.ataqueBase + (armaEquipada != null ?
armaEquipada.getBonusAtaque() : 0);
        notifier.notify("attack", this.nome, alvo.getNome());
        alvo.receberDano(ataqueTotal, notifier);
    }

    public List<Membro> getMembros() {
        return this.membros;
    }

    // ... demais métodos receberDano(), getNome(), etc.
}
```

O diagrama de classe

None

```
classDiagram
    class Destrutivel {
        <<interface>>
        +receberDano(valor: int, notifier: CombatNotifier) void
        +estaDestruido() boolean
        +getNome() String
    }

    class Personagem {
        <<abstract>>
        #String nome
        #int vida
        #int ataqueBase
        +equiparArma(novaArma: Arma) void
        +droparArmaAoMorrer() Arma
        +atacar(alvo: Destrutivel) void
    }

    class Membro {
        -String nome
        -int integridade
```

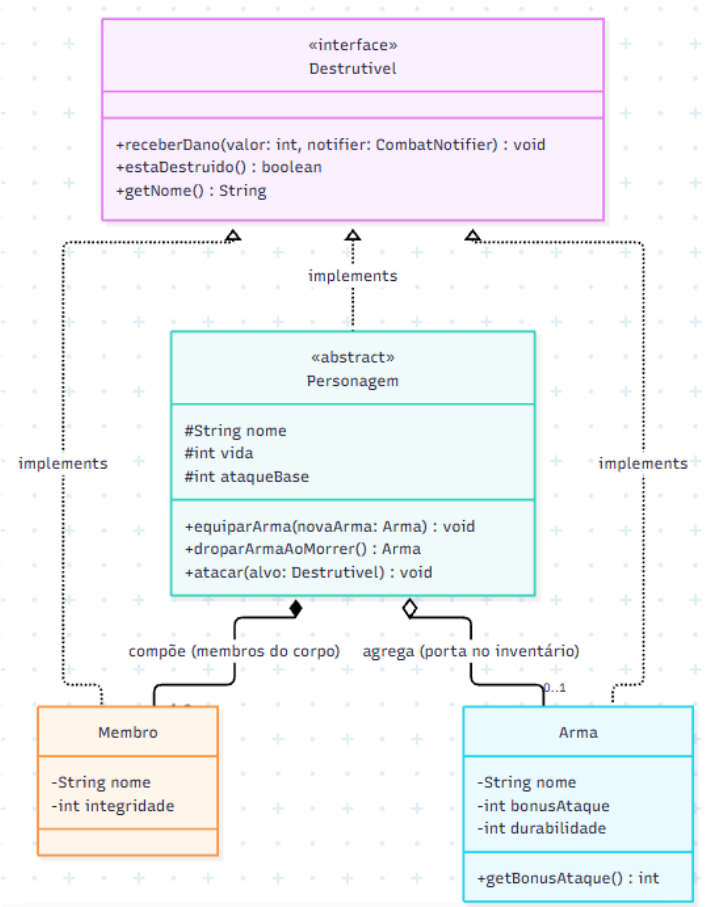
```
}

class Arma {
    -String nome
    -int bonusAtaque
    -int durabilidade
    +getBonusAtaque() int
}

Destrutivel <|.. Personagem : implements
Destrutivel <|.. Membro : implements
Destrutivel <|.. Arma : implements

%% COMPOSIÇÃO: Losango preenchido (membros morrem com o Personagem)
Personagem *-- "1..*" Membro : compõe (membros do corpo)

%% AGREGAÇÃO: Losango vazio (arma existe fora do Personagem)
Personagem o-- "0..1" Arma : agrega (porta no inventário)
```



Outras Ideias Complementares Válidas

Caso queira variar ou enriquecer os exemplos além de membros do corpo e armas:

1. **Agregação (Todo-Parte Fraco):**
    - **`GrupoDeBatalha` agregando `Personagem`:** Se o grupo for desfeito, os personagens continuam existindo vivos.
    - **`Bau` agregando `Item`:** Quando o baú quebra (`Bau.receberDano`), os itens caem no chão; a destruição do baú não aniquila os itens contidos nele.
2. **Composição (Todo-Parte Forte):**
    - **`Fortaleza` composta por `Muralha` e `Portao`:** O "Portão da Cidadela" é parte estrutural da fortaleza específica. Derrubada a fortaleza, suas seções estruturais deixam de existir como entidade independente.
    - **`LivroDeMagias` composto por `Pagina`:** Uma página pertence àquele tomo físico específico; destruído o livro, as páginas se perdem.

Código para experimentação das associações

Java

```
import java.text.MessageFormat;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// ========================================
// 1. CONTRATOS & I18N / SAÍDA
// ========================================
interface MessageProvider {
    String getMessage(String key, Object... args);
}

class PortugueseMessageProvider implements MessageProvider {
    private final Map<String, String> messages = new HashMap<>();

    public PortugueseMessageProvider() {
        messages.put("attack", "{0} ataca {1} causando {2} de dano!");
          messages.put("damage_received", "{0} recebeu {1} de dano (Restante:
{2}).");
        messages.put("defeated", "{0} foi derrotado!");
        messages.put("weapon_equipped", "{0} equipou [{1}] (+{2} de ATQ).");
         messages.put("weapon_dropped", "{0} foi derrotado e derrubou [{1}] no
chão!");
                      messages.put("limb_severed",   "CRÍTICO!   O   {0}   foi
decepado/destruído!");
    }
```

```
    @Override
    public String getMessage(String key, Object... args) {
        return MessageFormat.format(messages.getOrDefault(key, key), args);
    }
}

interface OutputChannel {
    void print(String message);
}

class ConsoleOutputChannel implements OutputChannel {
    @Override
    public void print(String message) {
        System.out.println(message);
    }
}

class CombatNotifier {
    private final OutputChannel output;
    private final MessageProvider messages;

    public CombatNotifier(OutputChannel output, MessageProvider messages) {
        this.output = output;
        this.messages = messages;
    }

    public void notify(String key, Object... args) {
        output.print(messages.getMessage(key, args));
    }
}

interface Destrutivel {
    void receberDano(int valor, CombatNotifier notifier);
    boolean estaDestruido();
    String getNome();
}

// ========================================
// 2. PARTE DA COMPOSIÇÃO: MEMBRO
// ========================================
class Membro implements Destrutivel {
    private final String nome;
    private int integridade;

    public Membro(String nome, int integridade) {
        this.nome = nome;
        this.integridade = integridade;
    }
```

```
    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.integridade = Math.max(0, this.integridade - valor);
                        notifier.notify("damage_received",   this.nome,   valor,
this.integridade);
        if (estaDestruido()) {
            notifier.notify("limb_severed", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.integridade == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}

// ==========================================
// 3. PARTE DA AGREGAÇÃO: ARMA
// ==========================================
class Arma implements Destrutivel {
    private final String nome;
    private final int bonusAtaque;
    private int durabilidade;

    public Arma(String nome, int bonusAtaque, int durabilidade) {
        this.nome = nome;
        this.bonusAtaque = bonusAtaque;
        this.durabilidade = durabilidade;
    }

    public int getBonusAtaque() {
        return this.bonusAtaque;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.durabilidade = Math.max(0, this.durabilidade - valor);
                        notifier.notify("damage_received",   this.nome,   valor,
this.durabilidade);
    }

    @Override
    public boolean estaDestruido() {
        return this.durabilidade == 0;
```

```java
    }

    @Override
    public String getNome() {
        return this.nome;
    }
}

// ========================================
// 4. PERSONAGEM (TODO: COMPOSIÇÃO E AGREGAÇÃO)
// ========================================
abstract class Personagem implements Destrutivel {
    protected String nome;
    protected int vida;
    protected int ataqueBase;

        // COMPOSIÇÃO: Membros criados e geridos estritamente dentro do Personagem
    protected final List<Membro> membros = new ArrayList<>();

    // AGREGAÇÃO: Arma externa associada opcionalmente (pode virar loot)
    protected Arma armaEquipada;

    public Personagem(String nome, int vida, int ataqueBase) {
        this.nome = nome;
        this.vida = vida;
        this.ataqueBase = ataqueBase;

        // Instanciação forte da Composição
        this.membros.add(new Membro("Braço de " + nome, 12));
        this.membros.add(new Membro("Perna de " + nome, 18));
    }

    // Gerenciamento da Agregação
    public void equiparArma(Arma novaArma, CombatNotifier notifier) {
        this.armaEquipada = novaArma;
            notifier.notify("weapon_equipped", this.nome, novaArma.getNome(), novaArma.getBonusAtaque());
    }

    public Arma droparArmaAoMorrer(CombatNotifier notifier) {
        if (this.armaEquipada != null) {
            Arma dropada = this.armaEquipada;
            this.armaEquipada = null; // Quebra a associação
            notifier.notify("weapon_dropped", this.nome, dropada.getNome());
            return dropada; // Retorna a referência viva da arma
        }
        return null;
    }
```

```
    public void atacar(Destrutivel alvo, CombatNotifier notifier) {
         int bonus = (armaEquipada != null) ? armaEquipada.getBonusAtaque() :
0;
        int danoTotal = this.ataqueBase + bonus;
        notifier.notify("attack", this.nome, alvo.getNome(), danoTotal);
        alvo.receberDano(danoTotal, notifier);
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.vida = Math.max(0, this.vida - valor);
        notifier.notify("damage_received", this.nome, valor, this.vida);
        if (estaDestruido()) {
            notifier.notify("defeated", this.nome);
        }
    }

    @Override
    public boolean estaDestruido() {
        return this.vida == 0;
    }

    @Override
    public String getNome() {
        return this.nome;
    }

    public List<Membro> getMembros() {
        return this.membros;
    }
}

class Guerreiro extends Personagem {
    public Guerreiro(String nome, int vida, int ataqueBase) {
        super(nome, vida, ataqueBase);
    }
}

class Inimigo extends Personagem {
    public Inimigo(String nome, int vida, int ataqueBase) {
        super(nome, vida, ataqueBase);
    }
}

// ========================================
// 5. EXECUÇÃO NO MAIN
// ========================================
public class Main {
```

```
    public static void main(String[] args) {
                        CombatNotifier notifier = new CombatNotifier(new
ConsoleOutputChannel(), new PortugueseMessageProvider());

        Guerreiro mike = new Guerreiro("Mike", 30, 6);
        Guerreiro dustin = new Guerreiro("Dustin", 25, 4);
        Inimigo demogorgon = new Inimigo("Demogorgon", 50, 15);

        // Instanciação da Arma antes de pertencer a qualquer personagem
        Arma espadaLendaria = new Arma("Espada Vorpal", 10, 50);

        System.out.println("=== 1. TESTE DE AGREGAÇÃO (Equipar Arma) ===");
        mike.equiparArma(espadaLendaria, notifier);

         System.out.println("\n=== 2. TESTE DE COMPOSIÇÃO (Atacando um Membro
Específico) ===");
        // Mike ataca o braço do monstro (uma parte da Composição)
        Destrutivel bracoDoMonstro = demogorgon.getMembros().get(0);
         mike.atacar(bracoDoMonstro, notifier); // Dano: 6 base + 10 da arma =
16 (decepa o braço de 12 HP!)

             System.out.println("\n=== 3. MORTE DO PORTADOR E DROP DA ARMA
(Agregação) ===");
        // O monstro contra-ataca fatalmente o herói
        demogorgon.atacar(mike, notifier);
        demogorgon.atacar(mike, notifier); // Vida de Mike zera

          // Mike morre e larga a arma; o objeto Arma continua existindo na
memória
        Arma armaNoChao = mike.droparArmaAoMorrer(notifier);

        System.out.println("\n=== 4. REUTILIZAÇÃO DO OBJETO AGREGADO ===");
        // Dustin encontra a arma caída e a equipa
        if (armaNoChao != null) {
            dustin.equiparArma(armaNoChao, notifier);
            dustin.atacar(demogorgon, notifier);
        }
    }
}
```

```
=== 1. TESTE DE AGREGAÇÃO (Equipar Arma) ===
Mike equipou [Espada Vorpal] (+10 de ATQ).

=== 2. TESTE DE COMPOSIÇÃO (Atacando um Membro Específico) ===
Mike ataca Braço de Demogorgon causando 16 de dano!
Braço de Demogorgon recebeu 16 de dano (Restante: 0).
CRÍTICO! O Braço de Demogorgon foi decepado/destruído!

=== 3. MORTE DO PORTADOR E DROP DA ARMA (Agrega◆ão) ===
Demogorgon ataca Mike causando 15 de dano!
Mike recebeu 15 de dano (Restante: 15).
Demogorgon ataca Mike causando 15 de dano!
Mike recebeu 15 de dano (Restante: 0).
Mike foi derrotado!
Mike foi derrotado e derrubou [Espada Vorpal] no chão!

=== 4. REUTILIZAÇÃO DO OBJETO AGREGADO ===
Dustin equipou [Espada Vorpal] (+10 de ATQ).
Dustin ataca Demogorgon causando 14 de dano!
Demogorgon recebeu 14 de dano (Restante: 36).
```

Pacotes (packages) java

Por uma questão de logística didática, não utilizamos pacotes (packages) aqui. A intenção com isso foi termos sempre todo o código presente em um único arquivo para facilitar a discussão didática.

No estágio atual, todas as interfaces, classes de domínio, adaptadores de saída e dicionários de idiomas convivem no mesmo arquivo ou no pacote raiz (default package), o que cria ruído e enfraquece o isolamento que os princípios SOLID buscam garantir.

Uma ideia de reestruturação

```
com.rpg/
│
├── domain/                 // O núcleo de regras puras do jogo
│   ├── model/           // Entidades: Personagem, Guerreiro, Mago, Inimigo
│   ├── equipment/       // Partes de agregação/composição: Arma, Membro
│   ├── contract/        // Interfaces: Destrutivel, ComHabilidadeEspecial
│   └── event/              // Contratos de notificação do combate
│
├── i18n/                   // Internacionalização
│   ├── MessageProvider.java     // Interface de tradução
│   ├── PortugueseMessages.java
│   ├── EnglishMessages.java
│   └── ChineseMessages.java
│
├── presentation/              // I/O e Canais de exibição
│   ├── OutputChannel.java       // Interface do canal
│   ├── ConsoleOutputChannel.java
│   └── LogInMemoryOutputChannel.java
│
└── app/                // Ponto de entrada e montagem (Composition Root)
    └── Main.java
```

Alguns benefícios

- Qualquer desenvolvedor que abrir o projeto entende de imediato onde mexer: para adicionar um idioma, abre i18n; para criar uma nova criatura, abre domain.model.
- No código em arquivo único, muitas classes precisam ser públicas ou protegidas para se comunicarem. Com pacotes bem delineados:
    - Métodos e classes auxiliares de cálculo interno (como detalhes da mecânica de regeneração ou estruturas de dicionários) não precisam de public.
- A estrutura em pacotes favorece a modularização e o trabalho distribuído em equipes ou ainda abrir para a comunidade algum ramo específico.
    - Podemos ter uma equipe de desenvolvedores para a parte gráfica, outra cuidando de upgrades no RPG acrescentando novas funcionalidades etc.
    - Podemos tornar o pacote i18n uma biblioteca externa ou ser empacotado em um arquivo JAR à parte, permitindo que terceiros criem pacotes de tradução (mods comunitários) sem ter que recompilar as regras centrais de combate do motor do jogo.

Por ora, penso em deixar isso por conta de vocês.

Mesclando paradigmas

Java

```java
import java.text.MessageFormat;
import java.util.*;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;

// ============================================================================
// 1. NÚCLEO ORIENTADO A OBJETOS (POO + SOLID)
// ============================================================================
interface MessageProvider {
    String getMessage(String key, Object... args);
}

class PortugueseMessageProvider implements MessageProvider {
    private final Map<String, String> messages = new HashMap<>();

    public PortugueseMessageProvider() {
        messages.put("attack", "{0} ataca {1} causando {2} de dano!");
        messages.put("damage_received", "{0} recebeu {1} de dano (Restante: {2}).");
        messages.put("defeated", "{0} foi derrotado!");
        messages.put("weapon_equipped", "{0} equipou [{1}].");
        messages.put("limb_severed", "CRÍTICO! {0} foi destruído!");
    }

    @Override
    public String getMessage(String key, Object... args) {
        return MessageFormat.format(messages.getOrDefault(key, key), args);
    }
}

interface OutputChannel {
    void print(String message);
}

class ConsoleOutputChannel implements OutputChannel {
    @Override
    public void print(String message) {
        System.out.println(message);
    }
}

class CombatNotifier {
    private final OutputChannel output;
    private final MessageProvider messages;

    public CombatNotifier(OutputChannel output, MessageProvider messages) {
        this.output = output;
        this.messages = messages;
    }

    public void notify(String key, Object... args) {
        output.print(messages.getMessage(key, args));
    }
}

interface Destrutivel {
    void receberDano(int valor, CombatNotifier notifier);
    boolean estaDestruido();
    String getNome();
    int getVida();
```

```java
}

class Membro implements Destrutivel {
    private final String nome;
    private int integridade;

    public Membro(String nome, int integridade) {
        this.nome = nome;
        this.integridade = integridade;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.integridade = Math.max(0, this.integridade - valor);
        notifier.notify("damage_received", this.nome, valor, this.integridade);
        if (estaDestruido()) {
            notifier.notify("limb_severed", this.nome);
        }
    }

    @Override public boolean estaDestruido() { return this.integridade == 0; }
    @Override public String getNome() { return this.nome; }
    @Override public int getVida() { return this.integridade; }
}

class Arma {
    private final String nome;
    private final int bonusAtaque;

    public Arma(String nome, int bonusAtaque) {
        this.nome = nome;
        this.bonusAtaque = bonusAtaque;
    }
    public int getBonusAtaque() { return this.bonusAtaque; }
    public String getNome() { return this.nome; }
}

abstract class Personagem implements Destrutivel {
    protected final String nome;
    protected final int vida;
    protected final int ataqueBase;
    protected final List<Membro> membros = new ArrayList<>();
    protected Arma armaEquipada;

    public Personagem(String nome, int vida, int ataqueBase) {
        this.nome = nome;
        this.vida = vida;
        this.ataqueBase = ataqueBase;
        this.membros.add(new Membro("Membro de " + nome, 10));
    }

    public void equiparArma(Arma arma, CombatNotifier notifier) {
        this.armaEquipada = arma;
        notifier.notify("weapon_equipped", this.nome, arma.getNome());
    }

    public void atacar(Destrutivel alvo, CombatNotifier notifier) {
        int bonus = (armaEquipada != null) ? armaEquipada.getBonusAtaque() : 0;
        int total = this.ataqueBase + bonus;
        notifier.notify("attack", this.nome, alvo.getNome(), total);
        alvo.receberDano(total, notifier);
    }
```

```
    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        int novaVida = Math.max(0, this.vida - valor);
        notifier.notify("damage_received", this.nome, valor, novaVida);
        if (novaVida <= 0) {
            notifier.notify("defeated", this.nome);
        }
    }

    @Override public boolean estaDestruido() { return this.vida == 0; }
    @Override public String getNome() { return this.nome; }
    @Override public int getVida() { return this.vida; }
    public int getAtaqueBase() { return this.ataqueBase; }

    public abstract Personagem comVida(int novaVida);
}

class Guerreiro extends Personagem {
    public Guerreiro(String nome, int vida, int ataqueBase) {
        super(nome, vida, ataqueBase);
    }
    @Override
    public Personagem comVida(int novaVida) {
        Guerreiro g = new Guerreiro(this.nome, novaVida, this.ataqueBase);
        g.armaEquipada = this.armaEquipada;
        return g;
    }
}

class Mago extends Personagem {
    public Mago(String nome, int vida, int ataqueBase) {
        super(nome, vida, ataqueBase);
    }
    @Override
    public Personagem comVida(int novaVida) {
        Mago m = new Mago(this.nome, novaVida, this.ataqueBase);
        m.armaEquipada = this.armaEquipada;
        return m;
    }
}

class Inimigo extends Personagem {
    private int vidaMutavel;

    public Inimigo(String nome, int vida, int ataqueBase) {
        super(nome, vida, ataqueBase);
        this.vidaMutavel = vida;
    }

    @Override
    public void receberDano(int valor, CombatNotifier notifier) {
        this.vidaMutavel = Math.max(0, this.vidaMutavel - valor);
        notifier.notify("damage_received", this.nome, valor, this.vidaMutavel);
        if (estaDestruido()) {
            notifier.notify("defeated", this.nome);
        }
    }

    @Override public boolean estaDestruido() { return this.vidaMutavel == 0; }
    @Override public int getVida() { return this.vidaMutavel; }

    @Override
    public Personagem comVida(int novaVida) {
```

```
        return new Inimigo(this.nome, novaVida, this.ataqueBase);
    }
}

// ============================================================================
// 2. PARADIGMA ESTRUTURADO / PROCEDURAL
// ============================================================================
class ModuloEstruturado {

    public static List<Personagem> gerarPelotao(int qtdGuerreiros, int qtdMagos) {
        List<Personagem> exercito = new ArrayList<>();
        int id = 1;

        for (int i = 0; i < qtdGuerreiros; i++) {
            exercito.add(new Guerreiro("Guerreiro-Recruta-" + id, 30, 8));
            id++;
        }

        for (int j = 0; j < qtdMagos; j++) {
            exercito.add(new Mago("Mago-Aprendiz-" + id, 20, 12));
            id++;
        }

        return exercito;
    }

    public static void executarAtaqueEmGrupo(List<Personagem> atacantes, Destrutivel alvo,
CombatNotifier notifier) {
        int i = 0;
        int totalAtacantes = atacantes.size();

        while (i < totalAtacantes && !alvo.estaDestruido()) {
            Personagem combatente = atacantes.get(i);
            combatente.atacar(alvo, notifier);
            i++;
        }
    }
}

// ============================================================================
// 3. PARADIGMA FUNCIONAL
// ============================================================================
class ModuloFuncional {

    // Pipeline funcional puro: recebe lista imutável e devolve novos objetos curados e fortificados
    public static List<Personagem> curarEFortificar(List<Personagem> combatentes, int corteDeVida, int
multiplicador) {
        Predicate<Personagem> precisaDeCura = p -> p.getVida() < corteDeVida;
        Function<Personagem, Personagem> triplicarVida = p -> p.comVida(p.getVida() * multiplicador);

        return combatentes.stream()
                .filter(precisaDeCura)
                .map(triplicarVida)
                .collect(Collectors.toUnmodifiableList());
    }
}
```

```java
// ============================================================================
// 4. PARADIGMA LÓGICO
// ============================================================================
class ModuloLogico {

    public record Fato(String predicado, String arg1, String arg2) {}
    public record RegraDeAfinidade(Fato condicao, String efeitoMultiplicador) {}

    public static class BaseDeConhecimento {
        private final Set<Fato> fatos = new HashSet<>();
        private final List<RegraDeAfinidade> regras = new ArrayList<>();

        public void declararFato(String predicado, String arg1, String arg2) {
            fatos.add(new Fato(predicado.toLowerCase(), arg1.toLowerCase(), arg2.toLowerCase()));
        }

        public void declararRegra(Fato condicao, String efeito) {
            regras.add(new RegraDeAfinidade(condicao, efeito));
        }

        public double inferirMultiplicadorDano(String elementoAtaque, String tipoInimigo) {
            String el = elementoAtaque.toLowerCase();
            String tipo = tipoInimigo.toLowerCase();

            for (RegraDeAfinidade regra : regras) {
                Fato cond = regra.condicao();
                if (cond.predicado().equals("fraqueza") && cond.arg1().equals(el) &&
cond.arg2().equals(tipo)) {
                    if (regra.efeitoMultiplicador().equals("CRITICO")) return 2.0;
                    if (regra.efeitoMultiplicador().equals("RESISTENTE")) return 0.5;
                }
            }
            return 1.0;
        }
    }
}



// ============================================================================
// 5. INTEGRAÇÃO DOS 4 PARADIGMAS NO FLUXO PRINCIPAL
// ============================================================================
public class Main {
    public static void main(String[] args) {
        CombatNotifier notifier = new CombatNotifier(new ConsoleOutputChannel(), new
PortugueseMessageProvider());


        // ------------------------------------------------------------------
        // ETAPA 1: PARADIGMA ESTRUTURADO (Procedural / Imperativo)
        // ------------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("1. PARADIGMA ESTRUTURADO: Recrutamento e Ataque");
        System.out.println("================================================");
        List<Personagem> pelotao = ModuloEstruturado.gerarPelotao(2, 1);
        Destrutivel chefe = new Inimigo("Demogorgon", 60, 10);

        System.out.println("Pelotão estruturado gerado com " + pelotao.size() + " combatentes.");
        ModuloEstruturado.executarAtaqueEmGrupo(pelotao, chefe, notifier);
        System.out.println();
```

```java
        // ----------------------------------------------------------------
        // ETAPA 2: PARADIGMA FUNCIONAL (Declarativo / Sem efeitos colaterais)
        // ----------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("2. PARADIGMA FUNCIONAL: Cura por Filter & Map");
        System.out.println("================================================");
        List<Personagem> veteranos = List.of(
            new Guerreiro("Thorin", 15, 10), // Vida < 25 (será fortificado)
            new Mago("Gandalf", 40, 15),     // Vida >= 25 (ignorado no filtro)
            new Guerreiro("Boromir", 8, 9)    // Vida < 25 (será fortificado)
        );

        System.out.println("Antes da cura funcional:");
        veteranos.forEach(v -> System.out.println("- " + v.getNome() + " (Vida: " + v.getVida() +
")"));

        // Pipeline imutável: filter (vida < 25) -> map (vida * 3)
        List<Personagem> fortificados = ModuloFuncional.curarEFortificar(veteranos, 25, 3);

        System.out.println("\nNovos objetos gerados pelo pipeline (sem mutar os originais):");
        fortificados.forEach(f -> System.out.println("- " + f.getNome() + " (Vida Curada e
Fortificada: " + f.getVida() + ")"));
        System.out.println();



        // ----------------------------------------------------------------
        // ETAPA 3: PARADIGMA LÓGICO (Declarativo / Fatos e Inferência)
        // ----------------------------------------------------------------
        System.out.println("================================================");
        System.out.println("3. PARADIGMA LÓGICO: Base de Regras e Inferência");
        System.out.println("================================================");
        ModuloLogico.BaseDeConhecimento baseLogica = new ModuloLogico.BaseDeConhecimento();

        baseLogica.declararFato("fraqueza", "fogo", "demogorgon");
        baseLogica.declararRegra(new ModuloLogico.Fato("fraqueza", "fogo", "demogorgon"), "CRITICO");

        baseLogica.declararFato("fraqueza", "gelo", "golem");
        baseLogica.declararRegra(new ModuloLogico.Fato("fraqueza", "gelo", "golem"), "RESISTENTE");

        double modFogo = baseLogica.inferirMultiplicadorDano("fogo", "demogorgon");
        double modRaio = baseLogica.inferirMultiplicadorDano("raio", "demogorgon");
        System.out.println("Modificador inferido [Fogo contra Demogorgon]: " + modFogo + "x");
      System.out.println("Modificador inferido [Raio contra Demogorgon]: " + modRaio + "x (Neutro)");
    }
}
```

```
==================================================
1. PARADIGMA ESTRUTURADO: Recrutamento e Ataque
==================================================
Pelotão estruturado gerado com 3 combatentes.
Guerreiro-Recruta-1 ataca Demogorgon causando 8 de dano!
Demogorgon recebeu 8 de dano (Restante: 52).
Guerreiro-Recruta-2 ataca Demogorgon causando 8 de dano!
Demogorgon recebeu 8 de dano (Restante: 44).
Mago-Aprendiz-3 ataca Demogorgon causando 12 de dano!
Demogorgon recebeu 12 de dano (Restante: 32).

==================================================
2. PARADIGMA FUNCIONAL: Cura por Filter & Map
==================================================
Antes da cura funcional:
- Thorin (Vida: 15)
- Gandalf (Vida: 40)
- Boromir (Vida: 8)

Novos objetos gerados pelo pipeline (sem mutar os originais):
- Thorin (Vida Curada e Fortificada: 45)
- Boromir (Vida Curada e Fortificada: 24)

==================================================
3. PARADIGMA LÓGICO: Base de Regras e Inferência
==================================================
Modificador inferido [Fogo contra Demogorgon]: 2.0x
Modificador inferido [Raio contra Demogorgon]: 1.0x (Neutro)
```

Discussão Didática sobre cada Paradigma Implementado

1. Paradigma Estruturado (Procedural)

- **Como foi implementado:** No ModuloEstruturado, usamos procedimentos puramente estáticos (gerarPelotao e executarAtaqueEmGrupo), contadores explícitos (id, i), estruturas sequenciais (ArrayList) e laços de repetição imperativos (for, while).
- **Foco Didático:** O programador diz explicitamente **como** a máquina deve proceder a cada passo da memória (alocação → incremento de contador → verificação de sentinela → avanço de índice).
- **Contraste com POO:** Não há encapsulamento de estado nem identidade de longo prazo no módulo; os dados são recebidos, processados em esteira linear e devolvidos/consumidos imediatamente.

2. Paradigma Funcional

- **Como foi implementado:** No ModuloFuncional, a sub-rotina curarEFortificar utiliza a Streams API do Java (filter e map), lambdas/interfaces funcionais (Predicate, Function) e o método de cópia comVida(...).
- **Foco Didático:** O programador declara **o que** deseja que aconteça com a coleção de dados, em vez de ditar como iterar sobre ela. Destacam-se dois pilares essenciais:
    1. **Ausência de efeitos colaterais (*pure functions*):** A lista original de veteranos permanece intocada.
    2. **Imutabilidade:** O map não altera campos de objetos existentes via setters; ele instancia novas representações com o estado transformado.

3. Paradigma Lógico

- **Como foi implementado:** No ModuloLogico, criamos uma base de conhecimento declarativa composta por **Fatos** (dados atômicos relacionais expressos via record Fato) e **Regras de Inferência** (RegraDeAfinidade), consultadas por um mecanismo de unificação e resolução (inferirMultiplicadorDano).
- **Foco Didático:** O código não prescreve nem a sequência passo a passo (estruturado) nem a transformação de fluxo (funcional). Ele declara axiomas do universo do jogo (*"Fogo é forte contra Demogorgon"*) e delega a dedução da resposta a um motor de consultas baseado em casamento de padrões (*pattern matching*).

Análise do trecho sob o paradigma estruturado

```
class ModuloEstruturado {

    public static List<Personagem> gerarPelotao(int qtdGuerreiros, int qtdMagos) {
        List<Personagem> exercito = new ArrayList<>();
        int id = 1;

        for (int i = 0; i < qtdGuerreiros; i++) {
            exercito.add(new Guerreiro("Guerreiro-Recruta-" + id, 30, 8));
            id++;
        }

        for (int j = 0; j < qtdMagos; j++) {
            exercito.add(new Mago("Mago-Aprendiz-" + id, 20, 12));
            id++;
        }

        return exercito;
    }
```

`public static List<Personagem> gerarPelotao(...)`

- **`static` (Sem estado):** O método pertence à classe utilitária, e não a uma instância (`new ModuloEstruturado()` não é necessário). Ele funciona como uma função pura ou sub-rotina da programação estruturada tradicional: recebe entradas, processa e devolve uma saída.
- **Entradas (`int qtdGuerreiros, int qtdMagos`):** Dois inteiros que ditam quantas repetições cada etapa fará.
- **Retorno (`List<Personagem>`):** Uma lista homogênea do tipo base, aproveitando o polimorfismo (aceita tanto `Guerreiro` quanto `Mago`).

`List<Personagem> exercito = new ArrayList<>();`

- Inicializa um vetor dinâmico vazio na memória que servirá como acumulador sequencial dos dados.

`int id = 1;`

- Uma **variável de controle escalar** (típica do paradigma imperativo) que atua como contador global da função para garantir que cada combatente receba um identificador numérico único e sequencial.

Primeiro laço: criação dos guerreiros

- O contador `i` comanda a repetição de 0 até $qtdGuerreiros - 1$.
- A cada ciclo, instancia um `Guerreiro` com vida fixa 30 e ataque base 8, concatenando o `id` atual ao nome.
- Incrementa `id` logo em seguida para o próximo personagem.

Segundo laço: criação dos magos

- Segue a mesma lógica sequencial imperativa, porém criando instâncias de Mago (vida 20, ataque 12).
- Como a variável id não foi reiniciada, ela continua a contagem de onde o laço anterior parou (ex.: se foram criados 2 guerreiros com IDs 1 e 2, o primeiro mago recebe o ID 3).

return exercito;

- Devolve a estrutura preenchida e encerra a sub-rotina.

Por que esse trecho é um exemplo clássico de "Paradigma Estruturado"?

Embora ele instancie classes de POO (`Guerreiro`, `Mago`), o **estilo de resolução do problema** é puramente estruturado:

- **Controle imperativo explícito ("como fazer"):** O desenvolvedor instrui a máquina passo a passo: inicialize uma variável, incremente um índice de 1 em 1, verifique a condição de parada e incremente um contador manual.
- **Execução sequencial em esteira:** O fluxo segue uma ordem estrita de cima para baixo: inicialização → bloco 1 → bloco 2 → retorno.
- **Ausência de estado encapsulado:** A classe `ModuloEstruturado` não guarda dados nem atributos de instância; ela funciona como um agrupador de procedimentos (equivalente a um arquivo `.c` ou módulo

Para fechar o raciocínio comparativo entre paradigmas, falta analisar a segunda sub-rotina do `ModuloEstruturado`: o método de ataque em grupo.

```
    public static void executarAtaqueEmGrupo(List<Personagem> atacantes, Destrutivel alvo,
CombatNotifier notifier) {
        int i = 0;
        int totalAtacantes = atacantes.size();

        while (i < totalAtacantes && !alvo.estaDestruido()) {
            Personagem combatente = atacantes.get(i);
            combatente.atacar(alvo, notifier);
            i++;
        }
    }
}
```

1. Por que essa versão é puramente Estruturada?

- **Controle explícito de fluxo com sentinela:** Usa um laço `while` com duas condições booleanas ligadas por `&&` (limite da lista e estado de destruição do alvo).
- **Indexação manual:** Mantém um ponteiro `int i = 0` incrementado cirurgicamente (`i++`) a cada passo.
- **Coordenação externa:** Nem o `alvo` nem a lista de `atacantes` têm autonomia; a função estática é quem puxa elemento por elemento e aciona as ações sequencialmente.

2. Equivalente em POO Pura: Delegação e Mensagens

Em POO, uma coleção de personagens não deve ser percorrida de fora por uma função estática. O próprio grupo (ou pelotão) assume o comportamento de executar uma ação coletiva, abstraindo os detalhes de repetição:

```
// Dentro da classe Pelotao (ou GrupoDeBatalha):
public class Pelotao {
    private final List<Personagem> membros = new ArrayList<>();

    // O próprio objeto sabe como realizar uma investida coordenada:
    public void realizarInvestida(Destrutivel alvo, CombatNotifier notifier) {
        for (Personagem membro : this.membros) {
            if (alvo.estaDestruido()) {
                break; // Interrompe se o alvo já sucumbiu
            }
            membro.atacar(alvo, notifier);
        }
    }
}
```

Vantagem de POO: O código cliente apenas envia uma mensagem de alto nível: `pelotao.realizarInvestida(demogorgon, notifier);`. O chamador não precisa saber o tamanho da tropa nem como o loop é feito internamente.

3. Equivalente no Paradigma Funcional: Pipeline

No paradigma funcional, evita-se o `while` imperativo e o `break`. A execução de efeitos sobre um fluxo até que uma condição de parada seja atingida é expressa com `takeWhile`:

```
public class BatalhaFuncional {

    public static void executarAtaqueEmGrupo(List<Personagem> atacantes, Destrutivel alvo,
CombatNotifier notifier) {
        atacantes.stream()
                // Interrompe o consumo do fluxo assim que o alvo for destruído
                .takeWhile(p -> !alvo.estaDestruido())
                // Executa a ação de ataque para cada combatente elegível
                .forEach(p -> p.atacar(alvo, notifier));
    }
}
```

Vantagem Funcional: O operador `takeWhile` expressa declarativamente a regra: *"processe enquanto o alvo estiver de pé"*. Não há contadores, índices ou chamadas `get(i)`.

Analisando as vantagens/desvantagens nas mesclagens de paradigmas

Algumas mesclagens foram um avanço arquitetural claro, enquanto outras geraram apenas ruído e complexidade artificial.

Para entender isso sem dogmas, vale avaliar cada inserção separadamente:

Onde a mesclagem foi uma GRANDE VANTAGEM: O Paradigma Funcional

Substituir algoritmos manuais de varredura por pipelines funcionais (`stream().filter().map()`) no método `curarEFortificar` é **vantagem absoluta**.

- **Por que não fazer em POO tradicional?** Em POO clássica, faríamos um `for`, criaríamos uma lista auxiliar temporária, verificaríamos `if (p.getVida() < 25)` e chamaríamos mutadores (`p.setVida(...)`). Isso introduz mutabilidade de estado e verbosidade desnecessária.
- **O ganho real:** O paradigma funcional eliminou efeitos colaterais e reduziu 15 linhas de controle de iteração a uma expressão declarativa legível e imutável.
- **Veredito:** É exatamente por isso que linguagens como Java, C#, Kotlin e Python adotaram recursos funcionais. **Ninguém em produção hoje escreveria isso em POO pura imperativa.**

Onde a mesclagem foi um ERRO TÉCNICO (apesar de didático): O Paradigma Estruturado

Criar o `ModuloEstruturado` com `gerarPelotao` e `executarAtaqueEmGrupo` foi um **retrocesso em relação ao design orientado a objetos**.

- **Por que foi desvantajoso?**
  - Quebrou o encapsulamento: classes com métodos estáticos manipulando dados de outras classes são conhecidas na literatura de arquitetura como o *anti-pattern* **Anemic Domain Model** (Modelo de Domínio Anêmico) ou cirurgia de procedimentos em dados alheios.
  - Violou o princípio da responsabilidade: um `Pelotao` ou uma fábrica `Quartel` (em POO pura) teriam autonomia sobre sua composição e regras de ataque.
- **Veredito:** Em um projeto real de produção, **manter em POO seria infinitamente superior**. A versão estruturada serviu apenas como contraexemplo pedagógico para mostrar como se programava antes dos objetos.

Onde a mesclagem foi NEUTRA / QUESTIONÁVEL: O Paradigma Lógico

A criação do motor de regras declarativo (`ModuloLogico`) resolveu um problema real, mas pagou um preço alto pela escolha da tecnologia.

- **A vantagem que ele trouxe:** Externalizou regras que, se feitas com herança de POO pura, gerariam uma "explosão combinatória de classes" (`InimigoVulneravelAFogo`, `InimigoResistenteAGelo`) ou uma cadeia frágil de `if/else` e `switch`. A abordagem declarativa baseada em fatos é elegante e desacoplada.
- **A desvantagem da implementação:** Como o Java não possui um interpretador lógico nativo (ao contrário de linguagens como Mercury ou Prolog), tivemos que programar o próprio mecanismo de resolução na mão. Em um sistema comercial, a equipe teria que manter esse mini-motor de inferência.

- **Veredito:** O conceito lógico é brilhante para regras de afinidade, mas simular Prolog dentro do Java em projetos de porte médio adiciona uma camada de complexidade cognitiva desproporcional.

Síntese Comparativa dos Paradigmas

Paradigma	Responsabilidade no Sistema	Mecanismo Central	Vantagens	Desvantagens / Limitações
Orientado a Objetos (POO)	Modelagem de entidades, ciclo de vida e contratos polimórficos de combate.	Encapsulamento, herança, interfaces (Destrutivel) e composição (Membro).	Mapeamento intuitivo do mundo real; alto reuso via abstrações; conformidade com SOLID.	Risco de acoplamento excessivo e hierarquias infladas; verbosidade para operações puramente matemáticas.
Estruturado (Procedural)	Orquestração algorítmica em lote e automação sequencial imperativa.	Funções estáticas, laços for/while, variáveis temporárias e índices lineares.	Curva de aprendizado mínima; controle direto e transparente do fluxo de execução e memória.	Dificuldade de escala em sistemas grandes; espalhamento de regras de negócio sem coesão; alto risco de dependência de estado global.
Funcional	Filtragem, transformação e enriquecimento de conjuntos de dados.	*Pipelines* de Streams, funções de primeira classe, filter(), map() e imutabilidade.	Código conciso e expressivo; ausência de efeitos colaterais acidentais; facilidade inerente para concorrência e testes.	Alocação intensiva de novos objetos na memória (impacto no *Garbage Collector*); curva conceitual para quem só programa com mutação imperativa. As bibliotecas importadas são nativas do JDK.
Lógico	Consulta declarativa e dedução de afinidades, fraquezas e efeitos de combate.	Base de conhecimento (Fatos/Regras) e motor de unificação / resolução.	Regras de negócio externalizadas e declarativas; adicionar novas interações não exige novos if/else; código focado no *quê*, não no *como*.	Em linguagens como Java, requer a construção manual de um interpretador de regras; custo computacional de busca recursiva/unificação em bases densas. É possível usar bibliotecas externas (como o tuProlog), mas iria requerer tratamento de dependências, talvez inviabilizando o uso de um arquivo único ou a execução no onlineGDB.